Security Assessment Report

Generated: 2026-05-05 11:01
Report ID: 6cf5452a

Security Assessment Report

1. Executive Summary

A static security assessment was performed against the codebase located at /workspace/uploads/b7490d70c6b8 using automated analysis tools. The scan results did not identify any confirmed security vulnerabilities in the analyzed Python source file, and no Sonar issues were reported.

The Bandit scan completed successfully with no findings, and the Sonar scan returned 0 issues. The dependency security scan could not be completed because no requirements.txt file was present in the project, which limits visibility into thirdparty package risk.

Overall risk level: Low

While no codelevel vulnerabilities were detected in the scanned scope, the absence of dependency metadata means the assessment is incomplete with respect to supplychain and packagelevel risks.

2. Risk Overview

Findings by Severity

Critical: 0

High: 0

Medium: 0

Low: 0

Informational: 1

Risk Interpretation

Critical / High / Medium / Low: No confirmed application vulnerabilities were identified by the available automated scans.

Informational: The project lacks a requirements.txt file, preventing automated dependency vulnerability analysis. This does not indicate a direct vulnerability, but it reduces assurance that the environment is free from known vulnerable packages.

3. Detailed Findings

Informational Findings

Finding 1 — Missing Dependency Manifest (requirements.txt)

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A06: Vulnerable and Outdated Components

Affected Component:

/workspace/uploads/b7490d70c6b8

Dependency management / Python package inventory

Description:

The safety scan reported that no requirements.txt file was found in the project.

Without a dependency manifest, it is not possible to reliably enumerate and assess installed Python packages for known vulnerabilities.

This creates a visibility gap in the software supply chain review process and may allow outdated or vulnerable thirdparty libraries to remain undetected.

Impact:

Attackers may be able to exploit known vulnerabilities in unreviewed dependencies if they are present in the runtime environment.

From a business perspective, this increases operational and security risk, particularly if the application relies on external libraries for authentication, serialization, networking, or data handling.

It also reduces auditability and complicates patch management.

Proof of Concept (if possible):

Not directly exploitable as a standalone issue.

The practical concern is that automated dependency scanning cannot be performed without a

manifest, so vulnerable packages may go unnoticed.

Recommended Remediation:

- Add and maintain a requirements.txt file or equivalent dependency lockfile such as poetry.lock, Pipfile.lock, or uv.lock.
- Pin dependency versions where appropriate to improve reproducibility.
- Run regular dependency vulnerability scans in CI/CD using tools such as pipaudit, safety, or dependency management platform integration.
- Review and update thirdparty libraries on a scheduled basis.

4. Recommendations (Global)

- Maintain a complete dependency inventory, including all runtime and development packages.
- Integrate software composition analysis into the build pipeline to detect vulnerable or outdated libraries early.
- Continue secure coding practices such as:
- strict input validation,
- output encoding where applicable,
- least privilege,
- secure error handling,
- avoidance of hardcoded secrets.
- Perform periodic code reviews and automated SAST scans as part of release gates.
- Ensure secrets, credentials, and configuration values are stored outside source code and managed securely.
- Keep the application and its dependencies updated to supported versions.
- Expand testing coverage to include authentication, authorization, and edgecase abuse scenarios if the application exposes usercontrolled input or external interfaces.

5. Conclusion

The automated scan of the provided codebase did not identify any confirmed sourcecode vulnerabilities, which is a positive indicator of the current security posture. However, the assessment is limited by the absence of a dependency manifest, preventing full evaluation of thirdparty package risk.

Overall, the system currently appears to be at Low risk, but the dependency visibility gap

should be addressed promptly to ensure the environment is not exposed to known vulnerable components.